```
    FactoryBot.create(
      :task, project: project, title: "Use Telescope", size: 1,
             project_order: 2)
    FactoryBot.create(
      :task, project: project, title: "Take Notes", size: 1,
             project_order: 3)
    visit(project_path(project))
    find("#task_3")
    within("#task_3") do
      click_on("Up")
    end
    assert_selector(
      "tbody:nth-child(2) .name", text: "Take Notes")
    visit(project_path(project))
    find("#task_2")
    within("#task_2") do
      assert_selector(".name", text: "Take Notes")
    end
  end
end
```

It's very similar to the RSpec version, with two exceptions:

- Minitest doesn't have let, so I've pulled the object creation into the test.
- Rather than use matchers, Minitest uses assert_selector, which takes the same arguments.

Since the rest of the test uses Capybara, it uses the same calls as the RSpec version we've been using in the rest of the book.

Rails does not run system tests as part of a general rails test command, on the theory that system tests are slow and you might want to run them separately. The command to run the system tests is rails test:system, which will successfully run these tests.

Minitest Integration Tests

Rails uses integration tests for end-to-end tests when the runtime is not using JavaScript. These are a little different from the system tests in that they don't use Capybara, and therefore typically pass parameters directly to the test.

You have end-to-end tests that you used to create projects. Here's how those look as integration tests:

minitest/02/test/integration/add_project_test.rb

```
require "test_helper"

class AddProjectTest < ActionDispatch::IntegrationTest
  test "allows a user to create a project with tasks" do
    post(projects_path, params: {project:
```